AI-NATIVE STARTUP ORGANIZATION PLAYBOOK

A Complete Operations Manual from Day 1 to Daily Execution

JIALU CHENG

2026-04-22

AI-Native Startup Organization Playbook

作者： Jialu Cheng

出版： X\: @Xialu8888

版本： v3.0

日期： 2026-04-22

简介： Complete collaboration SOP for 5-person team with 16-hour timezone spread shipping product in 4 days. Async-first, Feishu Bot automation, and relay baton handoff protocol for China-US cross-timezone AI-native startups.

版权所有 © 2026 Jialu Cheng。保留所有权利。
未经书面许可，不得以任何形式复制或传播本书内容。

目录

AI-Native Startup Organization Playbook

Quick Start Guide

Part 1: Collaboration Infrastructure

§ 01 Core Philosophy: Async-First, AI-Coordinated

§ 02 Tool Stack: Feishu Ecosystem + AI Tools

§ 03 Knowledge Layer: Making Your Company Machine-Readable

§ 02 Tool Stack: Feishu Ecosystem + AI Tools

§ 03 Knowledge Layer: Making Your Company Machine-Readable

Part 2: Cross-Timezone Operations

§ 04 Cross-Timezone Operations: China-US Dual Timezone Mechanism

Part 2: Daily Operations SOP

§ 05 Daily Communication: Bot + Async Messages Replace 80% of Meetings

§ 06 Weekly Meetings & Decisions: How to Run the One Meeting Worth Having

Part 3: Pod Operations

§ 07 Pod Structure: Small Teams + AI Agents

Part 4: Business Workflows

§ 08 Product Workflow: 4-Day Cycle from Idea to Live

§ 09 Engineering Workflow: AI Pair Programming Daily

§ 10 Growth Workflow: 2 People Doing 20 People’s Work

§ 11 Customer & Operations Workflow: AI Agents on the Front Line

Part 4: Advanced Operations

§ 12 Onboarding: AI-Native 1-Week Ramp-Up

§ 13 Security Guardrails & Agent Permissions: Draw Lines on Day 1

§ 14 Scaling from 5 to 50: How Collaboration Evolves

Appendices

AI-NATIVE STARTUP ORGANIZATION PLAYBOOK

A Complete Operations Manual: From Day 1 to Daily Execution

TL;DR

- 5-person team, 16-hour timezone spread (UTC+8 ↔ UTC-7), shipped product in 4 days
- Secret: Async-first + Feishu (Lark) Bot automation + "relay baton" handoff protocol
- One meeting per week. Everything else async.
- 3x iteration speed vs traditional teams through AI-coordinated workflows

---

Author: Jialu Cheng (@Xialu8888)
For: Founders & core teams building AI-native startups (China-US cross-timezone)
Based on: 2025-2026 real-world practices from AI-native startups
Last Updated: 2026-04-22
Stack: Feishu (Lark) ecosystem, GitHub, Claude Code/Cursor, n8n

---

Quick Start Guide

TIMELINE	SECTIONS	WHAT TO BUILD
Day 1 AM	§ 01- § 03	Feishu stack + Knowledge base skeleton
Day 1 PM	§ 04- § 05	Cross-timezone collaboration mechanism + Daily comms SOP
Day 2	§ 06- § 07	Weekly meeting template + Pod operation rules
Day 3	§ 08- § 09	Product & engineering workflows live
Day 4	§ 10- § 11	Growth & operations workflows live
First Weekend	§ 12- § 14	Onboarding flow + Security guardrails

---

Part 1: Collaboration Infrastructure

Build this on Day 1. Skip it, and you’ll pay compound interest on technical debt.

---

§ 01 Core Philosophy: Async-First, AI-Coordinated

01.1 The Bottom Line

After studying enough AI-native startups, the pattern is clear:

They operate fundamentally differently from traditional companies.

Not because of tools. The underlying logic has shifted.

Traditional company flow:

```
Meeting alignment → Division of work → Individual execution → Sync meeting → Re-adjust
```

AI-native company flow:

```
Async communication → AI auto-coordination → Humans handle exceptions only → AI continuous sync
```

When your team spans China (UTC+8) and US West Coast (UTC-7/8), that's a 15-16 hour gap. You cannot rely on meetings.

> ***Core Insight**: Meetings are not collaboration. Meetings are collaboration failure.*
>
> *A 5-person AI startup across China-US runs **one meeting per week**. Everything else: async + Feishu Bot sync. Their product iteration speed is 3x traditional teams. Simple reason: When it's your morning, your teammate is sleeping. Without async, you don't move.*

01.2 Three Principles

Principle 1: Async-First

Default communication is async: Lark messages, document comments, recorded videos. Only sync when absolutely necessary.

Atlassian research: 72% of meetings are ineffective. Cross-timezone teams: even higher — add "meetings scheduled at inconvenient hours to accommodate timezones."

COMMUNICATION TYPE	TRADITIONAL	AI-NATIVE (FEISHU STACK)	RECOMMENDATION
Status sync	Daily standup	Feishu Bot auto-aggregate + async updates	Eliminate standups entirely
Info sharing	PowerPoint meeting	Lark Doc + 5-min Loom video	Teammate watches when they wake up
Design discussion	Meeting brainstorm	Doc comments + Bot summary	24-hour async window
Urgent issue	Emergency meeting	Urgent message + 15-min video call	Only sync scenario
Decision confirmation	Decision meeting	Doc voting + Bot tally	Async decisions are faster

Principle 2: AI handles information, humans handle judgment

Information aggregation, status sync, progress tracking, meeting notes, task assignment, data reporting — give all to AI (Feishu Bot).

Value judgment, direction decisions, client relationships, taste control, exception handling — keep for humans.

Principle 3: Everything traceable

Every decision, discussion, change has written record. No phone decisions, no mental storage. AI Agents and offline teammates need context to work.

Cross-timezone iron rule: Every decision you make must be understandable by a teammate waking up 12 hours later.

> ***Key Practice***: *Change "default to meeting" to "default to Lark Doc"*
>
> *When anyone suggests "let's have a meeting," ask: "Can this be solved with a document?" If yes, write the doc. @mention stakeholders for async comments. Bot summarizes. Done. Saves 1 hour. Cross-timezone: you write before sleep, they comment when waking — 3x more efficient than waiting for overlap.*

01.3 A Day in the Life: China-US AI-Native Team

```
                Beijing (UTC+8)        US West (UTC-7)
                ───────────────        ───────────────

08:00 Beijing    Work starts            17:00 (previous day)
                ├── Bot pushes "Beijing Morning Brief"
                ├── Review async updates from US overnight
                └── Reply to doc comments

09:00-12:00      Deep Work              18:00-21:00
                Beijing focused        US off-duty/rest

12:00 Beijing    Lunch                  21:00
                └── Bot pushes noon summary

13:00-17:00      Deep Work              22:00-02:00
                Continue work          US sleeping

17:00 Beijing    Before leaving         02:00
                ├── Write daily summary to doc
                ├── Document decisions/questions/blockers
                └── Bot auto-pushes "Beijing Daily Report"

── Relay Baton Handoff ──

00:00 Beijing    Sleeping               09:00 US starts
                                       ├── Bot pushes "US Morning Brief"
                                       ├── Review Beijing async updates
                                       └── Reply to doc comments

                                       09:00-17:00 US Deep Work

                                       17:00 US before leaving
                                       ├── Write daily summary
                                       └── Bot pushes "US Daily Report"

── Overlap Window (only sync time) ──

08:00-10:00 Beijing = 17:00-19:00 US (prev day)
└── Reserve for important decisions & urgent issues
└── Most weeks, you won't need this window
```

> ***Focus on***: *The relay baton handoff. China-US collaboration is a relay race. Beijing runs a leg, passes to US. US runs, passes back. Lark Docs and Bots are the baton. Key: When passing, context must be crystal clear. Never make them guess.*

§ 02 Tool Stack: Feishu Ecosystem + AI Tools

02.1 Why Feishu (Lark)

Three reasons for all-in Feishu:

1. **All-in-one**: Messages, docs, Base (smart tables), projects, calendar, video, Bot — all in one app. No context switching between 9 apps.
2. **API & Bot ecosystem**: Feishu’s open platform is powerful. Custom Bots push to groups, read/write docs, operate smart tables. Foundation for AI Agent integration.
3. **Works in both China and US**: Lark (international Feishu) works normally in the US. One system, two timezones.

02.2 Core Stack

LAYER	FEISHU TOOL	PURPOSE	AI AGENT INTEGRATION
Communication Hub	Lark Messages (Groups)	Daily comms + Bot notifications	Bot API pushes Daily Brief
Knowledge Base	Lark Docs	Decisions, PRDs, SOPs, meeting notes	Bot auto-creates docs, writes content
Data Center	Lark Base (多维表格)	Backlog, Bug tracking, OKRs, CRM	Bot auto reads/writes records, triggers automation
Project Mgmt	Lark Projects	Sprint management, Kanban	Bot auto-updates task status
Calendar	Lark Calendar	Meetings, milestones, deadlines	Bot auto-creates events & reminders
Video	Lark Meetings	Weekly meeting (1x/week), emergency huddles	AI Minutes auto-transcribe + summarize
Approvals	Lark Approvals	Budget, permissions, release approvals	Bot triggers and notifies

Only 3 non-Feishu tools needed:

TOOL	PURPOSE	WHY NOT FEISHU
GitHub	Code hosting, PRs, CI/CD	Industry standard for code
Claude Code / Cursor	AI pair programming	Feishu has no AI coding capability
n8n (self-hosted)	Workflow automation, Agent orchestration	Feishu automation can’t handle complex cross-system flows

> ***Key Point**: 10 tools total. 7 Feishu + 3 external. The 7 Feishu tools connect natively. External 3 connect via n8n and Feishu Bot API. Information never breaks.*

02.3 Feishu Group Structure

Groups aren't random. AI-native companies design information flow intentionally.

GROUP	PURPOSE	MEMBERS	BOT ROLE
#daily-brief	Auto daily summary	Everyone	Auto-generate & push (Beijing + US versions)
#decisions	All decision records	Everyone	Auto-archive to Lark Doc
#product	Product discussion	Product + Eng	Summarize discussions, extract todos to Base
#engineering	Technical discussion	Engineering	PR notifications, CI status pushes
#growth	Growth discussion	Growth	Data anomaly alerts
#customer	Customer feedback	Everyone	AI auto-classify, draft replies
#alerts	System alerts	Eng + Founders	Auto-push + urgent notification
#random	Water cooler	Everyone	No intervention

> ***Iron Rule**: No decisions in DMs*
>
> *All work discussions happen in groups or doc comments. DMs are invisible to Bots and teammates. Cross-timezone teammates definitely won't see them.*

02.4 Feishu Bot Setup

Create a custom Bot via Feishu Open Platform. This Bot is the nervous system of your AI collaboration.

What the Bot can do:

```
1. Push messages to groups (Daily Brief, alerts, notifications)
2. Read group messages (monitor discussions, extract key info)
3. Operate Lark Docs (auto-create, write content, read comments)
4. Operate Lark Base (auto-create records, update status, trigger notifications)
5. Operate Lark Projects (update task status)
6. Manage calendar (create events, send reminders)
```

Architecture:

```
n8n (Automation Engine)
    │
    ├── Scheduled Triggers
    │   ├── 08:00 Beijing → Generate Beijing Brief → Bot pushes
    │   ├── 09:00 US → Generate US Brief → Bot pushes
    │   └── 17:00 each timezone → Generate daily report → Bot pushes
    │
    ├── Webhook Triggers
    │   ├── GitHub PR events → Bot pushes to #engineering
    │   ├── Monitoring alerts → Bot pushes to #alerts (urgent)
    │   └── Customer tickets → AI classify → Bot pushes to #customer
    │
    └── AI Agent
        ├── Read doc comments → Summarize discussion → Push conclusion
        ├── Read Base data → Generate analysis → Write to doc
        └── Read key group messages → Extract todos → Write to Projects
```

> ***Setup time: ~3 days*** *Day 1: Create Feishu Bot + permissions + group structure. Day 2: Set up n8n + connect Feishu API + write first Daily Brief flow. Day 3: GitHub→Feishu notification flow + customer ticket flow. After 3 days, the system runs automatically. Highest ROI 3-day investment I've made.*

02.5 Lark Base (多维表格) Core Usage

Lark Base is your data hub. More structured than docs, easier than databases, Bot can read/write directly.

4 Bases to build on Day 1:

BASE	FIELDS	PURPOSE	BOT USAGE
Backlog	Title/Source/Priority/Status/Owner/Sprint	Full product lifecycle	AI auto-extract from customer feedback
Bug Tracker	Title/Severity/Status/Owner/Linked PR	Bug tracking	Sync from GitHub Issues
Customer CRM	Company/Contact/Stage/Last Touch/Next Step	Lightweight CRM	Bot auto-updates interaction records
Sprint Board	Task/Owner/Status/Pod/ETA	Current sprint execution	Auto-sync completion from GitHub

> ***Single Source of Truth*** *Don't say in chat "this bug is fixed". Change status to "Fixed" in Base. Bot notifies stakeholders. Cross-timezone teammates check Base, not chat history.*

---

§03 Knowledge Layer: Making Your Company Machine-Readable

03.1 Knowledge Layer ≠ "Writing Docs"

In early 2026, an AI founder friend said something I've been thinking about: "Our most valuable asset isn't code. It's the decisions and lessons in Feishu docs. Code can be rewritten by AI. That knowledge is the real asset."

What is the Knowledge Layer?

Everything — decisions, docs, meeting records, customer feedback, playbooks, lessons learned — in **structured, machine-readable** form in Lark Docs and Base, accessible to Bots.

> ***Hallway conversations are knowledge black holes***
>
> *Common scenario: Decisions made over lunch or walking. "Let's deprioritize this feature," "I'll handle that client." Information lives in two brains. Bot doesn't know. Teammates don't know. US teammates definitely don't know. AI-native orgs cannot have dark knowledge. Cross-timezone: your "hallway conversation" is permanently invisible to them.*

03.2 Build These 6 Spaces in Lark Docs

Create on Day 1. Use Feishu Knowledge Base structure.

```
 Company Knowledge Base (Feishu Knowledge Base)
├── 1-Decision Log
│   ├── Template: [Date] [Decision Title]
│   ├── Content: Decision / Rationale / Data / Decision-maker
│   └── Rule: Must log within 24 hours of decision
│
├── 2-Product
│   ├── PRD Template (Lark Doc)
│   ├── Product Roadmap (Lark Doc)
│   ├── User Feedback Library (Lark Base → Bot auto-aggregate)
│   └── Competitive Analysis (Lark Doc)
│
├── 3-Engineering
│   ├── Architecture Docs (Lark Doc)
│   ├── API Docs (Lark Doc)
│   ├── Postmortems (Lark Doc)
│   └── Code Standards (Lark Doc)
│
├── 4-SOPs (Playbooks)
│   ├── Deploy SOP / Release SOP / Incident SOP
│   ├── Customer Support SOP
│   └── Cross-timezone Collaboration SOP ← New, important
│
├── 5-Growth
│   ├── Channel Strategy (Lark Doc)
│   ├── Content Calendar (Lark Base)
│   ├── Data Dashboard (Lark Base dashboard)
│   └── Experiment Log (Lark Base)
│
└── 6-Company Ops
    ├── Team Directory + Timezone labels
    ├── New Hire Onboarding Guide
    ├── Tool Usage Guide
    └── Feishu Bot list & permissions
```

03.3 Five Rules

Rule 1: Lark Docs are primary, Markdown is backup

Daily collaboration in Lark Docs — comments, @mentions, co-editing. But export key docs to Markdown for GitHub, ensuring AI Agents can read.

Rule 2: Document the "why" for every decision

Decision log template:

```
## [2026-04-22] Pause Development of Feature X

**Decision**: Pause Feature X, reallocate resources to Feature Y
**Rationale**: User research shows only 3% need X, 67% need Y
**Data**: [Link to Lark Base]
**Decision-maker**: Jialu Cheng
**Impact**: Product Pod needs to reschedule
**Follow-up**: @Alice Update roadmap @Bob Notify customers
**Cross-timezone note**: US team please confirm engineering impact tomorrow morning
```

Rule 3: Every meeting must have Minutes (妙记)

All Lark Meetings enable Minutes. Auto-record + transcribe + AI summary + todo extraction. Those who didn't attend (teammates sleeping in other timezone) just read the Minutes.

Rule 4: Naming convention

`[Date]-[Type]-[Title]`. Clear, searchable doc titles.

Rule 5: Weekly knowledge health check

Every Friday, Bot auto-checks:

- How many decisions this week? Are they all logged?
- Any overdue tasks in Base?
- Any "dark knowledge" — discussed in chat but not documented?

> ***Cross-timezone knowledge requirement is 2x normal teams*** *You don't have "walk over and ask" privilege. Every word you write is the only context your cross-timezone teammate has. Clear writing = smooth collaboration. Vague writing = 12-hour back-and-forth confirmation = 1 day wasted.*

Not because of tools. The underlying logic has shifted.

Traditional company flow:

```
Meeting alignment → Division of work → Individual execution → Sync meeting → Re-adjust
```

AI-native company flow:

```
Async communication → AI auto-coordination → Humans handle exceptions only → AI continuous sync
```

When your team spans China (UTC+8) and US West Coast (UTC-7/8), that's a 15-16 hour gap. You cannot rely on meetings.

> ***Core Insight**: Meetings are not collaboration. Meetings are collaboration failure.*
>
> *A 5-person AI startup across China-US runs **one meeting per week**. Everything else: async + Feishu Bot sync. Their product iteration speed is 3x traditional teams. Simple reason: When it's your morning, your teammate is sleeping. Without async, you don't move.*

01.2 Three Principles

Principle 1: Async-First

Default communication is async: Lark messages, document comments, recorded videos. Only sync when absolutely necessary.

Atlassian research: 72% of meetings are ineffective. Cross-timezone teams: even higher — add "meetings scheduled at inconvenient hours to accommodate timezones."

COMMUNICATION TYPE	TRADITIONAL	AI-NATIVE (FEISHU STACK)	RECOMMENDATION
Status sync	Daily standup	Feishu Bot auto-aggregate + async updates	Eliminate standups entirely
Info sharing	PowerPoint meeting	Lark Doc + 5-min Loom video	Teammate watches when they wake up
Design discussion	Meeting brainstorm	Doc comments + Bot summary	24-hour async window
Urgent issue	Emergency meeting	Urgent message + 15-min video call	Only sync scenario
Decision confirmation	Decision meeting	Doc voting + Bot tally	Async decisions are faster

Principle 2: AI handles information, humans handle judgment

Information aggregation, status sync, progress tracking, meeting notes, task assignment, data reporting — give all to AI (Feishu Bot).

Value judgment, direction decisions, client relationships, taste control, exception handling — keep for humans.

Principle 3: Everything traceable

Every decision, discussion, change has written record. No phone decisions, no mental storage. AI Agents and offline teammates need context to work.

Cross-timezone iron rule: Every decision you make must be understandable by a teammate waking up 12 hours later.

> ***Key Practice***: *Change "default to meeting" to "default to Lark Doc"*
>
> *When anyone suggests "let's have a meeting," ask: "Can this be solved with a document?" If yes, write the doc. @mention stakeholders for async comments. Bot summarizes. Done. Saves 1 hour. Cross-timezone: you write before sleep, they comment when waking — 3x more efficient than waiting for overlap.*

01.3 A Day in the Life: China-US AI-Native Team

```
                Beijing (UTC+8)        US West (UTC-7)
                ───────────────        ───────────────

08:00 Beijing    Work starts            17:00 (previous day)
                ├── Bot pushes "Beijing Morning Brief"
                ├── Review async updates from US overnight
                └── Reply to doc comments

09:00-12:00      Deep Work              18:00-21:00
                Beijing focused        US off-duty/rest

12:00 Beijing    Lunch                  21:00
                └── Bot pushes noon summary

13:00-17:00      Deep Work              22:00-02:00
                Continue work          US sleeping

17:00 Beijing    Before leaving         02:00
                ├── Write daily summary to doc
                ├── Document decisions/questions/blockers
                └── Bot auto-pushes "Beijing Daily Report"

── Relay Baton Handoff ──

00:00 Beijing    Sleeping               09:00 US starts
                                       ├── Bot pushes "US Morning Brief"
                                       ├── Review Beijing async updates
                                       └── Reply to doc comments

                                       09:00-17:00 US Deep Work

                                       17:00 US before leaving
                                       ├── Write daily summary
                                       └── Bot pushes "US Daily Report"

── Overlap Window (only sync time) ──

08:00-10:00 Beijing = 17:00-19:00 US (prev day)
└── Reserve for important decisions & urgent issues
└── Most weeks, you won't need this window
```

> ***Focus on***: *The relay baton handoff. China-US collaboration is a relay race. Beijing runs a leg, passes to US. US runs, passes back. Lark Docs and Bots are the baton. Key: When passing, context must be crystal clear. Never make them guess.*

§ 02 Tool Stack: Feishu Ecosystem + AI Tools

02.1 Why Feishu (Lark)

Three reasons for all-in Feishu:

1. **All-in-one**: Messages, docs, Base (smart tables), projects, calendar, video, Bot — all in one app. No context switching between 9 apps.
2. **API & Bot ecosystem**: Feishu’s open platform is powerful. Custom Bots push to groups, read/write docs, operate smart tables. Foundation for AI Agent integration.
3. **Works in both China and US**: Lark (international Feishu) works normally in the US. One system, two timezones.

02.2 Core Stack

LAYER	FEISHU TOOL	PURPOSE	AI AGENT INTEGRATION
Communication Hub	Lark Messages (Groups)	Daily comms + Bot notifications	Bot API pushes Daily Brief
Knowledge Base	Lark Docs	Decisions, PRDs, SOPs, meeting notes	Bot auto-creates docs, writes content
Data Center	Lark Base (多维表格)	Backlog, Bug tracking, OKRs, CRM	Bot auto reads/writes records, triggers automation
Project Mgmt	Lark Projects	Sprint management, Kanban	Bot auto-updates task status
Calendar	Lark Calendar	Meetings, milestones, deadlines	Bot auto-creates events & reminders
Video	Lark Meetings	Weekly meeting (1x/week), emergency huddles	AI Minutes auto-transcribe + summarize
Approvals	Lark Approvals	Budget, permissions, release approvals	Bot triggers and notifies

Only 3 non-Feishu tools needed:

TOOL	PURPOSE	WHY NOT FEISHU
GitHub	Code hosting, PRs, CI/CD	Industry standard for code
Claude Code / Cursor	AI pair programming	Feishu has no AI coding capability
n8n (self-hosted)	Workflow automation, Agent orchestration	Feishu automation can’t handle complex cross-system flows

> ***Key Point**: 10 tools total. 7 Feishu + 3 external. The 7 Feishu tools connect natively. External 3 connect via n8n and Feishu Bot API. Information never breaks.*

02.3 Feishu Group Structure

Groups aren't random. AI-native companies design information flow intentionally.

GROUP	PURPOSE	MEMBERS	BOT ROLE
#daily-brief	Auto daily summary	Everyone	Auto-generate & push (Beijing + US versions)
#decisions	All decision records	Everyone	Auto-archive to Lark Doc
#product	Product discussion	Product + Eng	Summarize discussions, extract todos to Base
#engineering	Technical discussion	Engineering	PR notifications, CI status pushes
#growth	Growth discussion	Growth	Data anomaly alerts
#customer	Customer feedback	Everyone	AI auto-classify, draft replies
#alerts	System alerts	Eng + Founders	Auto-push + urgent notification
#random	Water cooler	Everyone	No intervention

> ***Iron Rule**: No decisions in DMs*
>
> *All work discussions happen in groups or doc comments. DMs are invisible to Bots and teammates. Cross-timezone teammates definitely won't see them.*

02.4 Feishu Bot Setup

Create a custom Bot via Feishu Open Platform. This Bot is the nervous system of your AI collaboration.

What the Bot can do:

```
1. Push messages to groups (Daily Brief, alerts, notifications)
2. Read group messages (monitor discussions, extract key info)
3. Operate Lark Docs (auto-create, write content, read comments)
4. Operate Lark Base (auto-create records, update status, trigger notifications)
5. Operate Lark Projects (update task status)
6. Manage calendar (create events, send reminders)
```

Architecture:

```
n8n (Automation Engine)
    │
    ├── Scheduled Triggers
    │   ├── 08:00 Beijing → Generate Beijing Brief → Bot pushes
    │   ├── 09:00 US → Generate US Brief → Bot pushes
    │   └── 17:00 each timezone → Generate daily report → Bot pushes
    │
    ├── Webhook Triggers
    │   ├── GitHub PR events → Bot pushes to #engineering
    │   ├── Monitoring alerts → Bot pushes to #alerts (urgent)
    │   └── Customer tickets → AI classify → Bot pushes to #customer
    │
    └── AI Agent
        ├── Read doc comments → Summarize discussion → Push conclusion
        ├── Read Base data → Generate analysis → Write to doc
        └── Read key group messages → Extract todos → Write to Projects
```

> ***Setup time: ~3 days*** *Day 1: Create Feishu Bot + permissions + group structure. Day 2: Set up n8n + connect Feishu API + write first Daily Brief flow. Day 3: GitHub→Feishu notification flow + customer ticket flow. After 3 days, the system runs automatically. Highest ROI 3-day investment I've made.*

02.5 Lark Base (多维表格) Core Usage

Lark Base is your data hub. More structured than docs, easier than databases, Bot can read/write directly.

4 Bases to build on Day 1:

BASE	FIELDS	PURPOSE	BOT USAGE
Backlog	Title/Source/Priority/Status/Owner/Sprint	Full product lifecycle	AI auto-extract from customer feedback
Bug Tracker	Title/Severity/Status/Owner/Linked PR	Bug tracking	Sync from GitHub Issues
Customer CRM	Company/Contact/Stage/Last Touch/Next Step	Lightweight CRM	Bot auto-updates interaction records
Sprint Board	Task/Owner/Status/Pod/ETA	Current sprint execution	Auto-sync completion from GitHub

> ***Single Source of Truth*** *Don't say "this bug is fixed" in chat. Change status to "Fixed" in Base. Bot notifies stakeholders. Cross-timezone teammates check Base, not chat history.*

---

§03 Knowledge Layer: Making Your Company Machine-Readable

03.1 Knowledge Layer ≠ "Writing Docs"

In early 2026, an AI founder friend said something I've been thinking about: "Our most valuable asset isn't code. It's the decisions and lessons in Feishu docs. Code can be rewritten by AI. That knowledge is the real asset."

What is the Knowledge Layer?

Everything — decisions, docs, meeting records, customer feedback, playbooks, lessons learned — in **structured, machine-readable** form in Lark Docs and Base, accessible to Bots.

> ***Hallway conversations are knowledge black holes***
>
> *Common scenario: Decisions made over lunch or walking. "Let's deprioritize this feature," "I'll handle that client." Information lives in two brains. Bot doesn't know. Teammates don't know. US teammates definitely don't know. AI-native orgs cannot have dark knowledge. Cross-timezone: your "hallway conversation" is permanently invisible to them.*

03.2 Build These 6 Spaces in Lark Docs

Create on Day 1. Use Feishu Knowledge Base structure.

```
 Company Knowledge Base (Feishu Knowledge Base)
├── 1-Decision Log
│   ├── Template: [Date] [Decision Title]
│   ├── Content: Decision / Rationale / Data / Decision-maker
│   └── Rule: Must log within 24 hours of decision
│
├── 2-Product
│   ├── PRD Template (Lark Doc)
│   ├── Product Roadmap (Lark Doc)
│   ├── User Feedback Library (Lark Base → Bot auto-aggregate)
│   └── Competitive Analysis (Lark Doc)
│
├── 3-Engineering
│   ├── Architecture Docs (Lark Doc)
│   ├── API Docs (Lark Doc)
│   ├── Postmortems (Lark Doc)
│   └── Code Standards (Lark Doc)
│
├── 4-SOPs (Playbooks)
│   ├── Deploy SOP / Release SOP / Incident SOP
│   ├── Customer Support SOP
│   └── Cross-timezone Collaboration SOP ← New, important
│
├── 5-Growth
│   ├── Channel Strategy (Lark Doc)
│   ├── Content Calendar (Lark Base)
│   ├── Data Dashboard (Lark Base dashboard)
│   └── Experiment Log (Lark Base)
│
└── 6-Company Ops
    ├── Team Directory + Timezone labels
    ├── New Hire Onboarding Guide
    ├── Tool Usage Guide
    └── Feishu Bot list & permissions
```

03.3 Five Rules

Rule 1: Lark Docs are primary, Markdown is backup

Daily collaboration in Lark Docs — comments, @mentions, co-editing. But export key docs to Markdown for GitHub, ensuring AI Agents can read.

Rule 2: Document the "why" for every decision

Decision log template:

```
## [2026-04-22] Pause Development of Feature X

**Decision**: Pause Feature X, reallocate resources to Feature Y
**Rationale**: User research shows only 3% need X, 67% need Y
**Data**: [Link to Lark Base]
**Decision-maker**: Jialu Cheng
**Impact**: Product Pod needs to reschedule
**Follow-up**: @Alice Update roadmap @Bob Notify customers
**Cross-timezone note**: US team please confirm engineering impact tomorrow morning
```

Rule 3: Every meeting must have Minutes (妙记)

All Lark Meetings enable Minutes. Auto-record + transcribe + AI summary + todo extraction. Those who didn't attend (teammates sleeping in other timezone) just read the Minutes.

Rule 4: Naming convention

`[Date]-[Type]-[Title]`. Clear, searchable doc titles.

Rule 5: Weekly knowledge health check

Every Friday, Bot auto-checks:

- How many decisions this week? Are they all logged?
- Any overdue tasks in Base?
- Any "dark knowledge" — discussed in chat but not documented?

> ***Cross-timezone knowledge requirement is 2x normal teams*** *You don't have "walk over and ask" privilege. Every word you write is the only context your cross-timezone teammate has. Clear writing = smooth collaboration. Vague writing = 12-hour back-and-forth confirmation = 1 day wasted.*

Part 2: Cross-Timezone Operations

§04 Cross-Timezone Operations: China-US Dual Timezone Mechanism

04.1 Timezone Gap ≠ Problem. Information Break = Problem.

Low efficiency in cross-timezone collaboration isn't caused by the timezone gap. It's caused by broken handoff protocols.

Typical scenario: Beijing makes a bunch of decisions during the day, leaves. US teammate opens Feishu in the morning: 20 group messages, no idea what's important, what's final, what they need to do. Spends 2 hours "archaeologizing" chat history to understand context.

A proper handoff system doubles efficiency immediately.

> ***Core of cross-timezone collaboration isn't "how to meet" — it's "how to handoff"*** *Like a relay race. When you finish your leg and pass the baton, ensure your teammate can immediately continue running. No need to wait for you to wake up and explain.*

04.2 Overlap Windows & Golden Hours

Beijing UTC+8 and US West UTC-7 (DST), 15-16 hour gap.

Available overlap:

BEIJING TIME	US WEST TIME	BEST USED FOR
08:00-10:00	17:00-19:00 (prev day)	Important decision discussions
21:00-23:00	06:00-08:00	Emergency issue alignment

> ***Reality check**: 4 hours max overlap, but only 2 hours truly usable. Don't waste these 2 hours on "sync progress" meetings — Feishu Bot syncs progress. Reserve for **must-discuss-face-to-face decisions and emergencies**. Most weeks, you won't even need this window.*

04.3 Relay Baton Handoff SOP

Most important SOP for cross-timezone collaboration.

Beijing team before leaving (17:00 UTC+8):

1. Write "Today's Handoff" in Lark Doc:
    - What was completed today (with links: PRs, docs, Base records)
    - What decisions were made (with rationale)
    - What US needs to do (clear tasks + deadlines)
    - What blockers need their help
    - Any context ("I chose Option A because XX, if you disagree, comment in doc")
2. Feishu Bot auto-generates "Beijing Daily" pushed to #daily-brief
3. Update all Base statuses to current

US team when starting (09:00 UTC-7):

1. Read Feishu Bot's "Beijing Daily" (3 minutes)
2. Read "Today's Handoff" in doc
3. Reply with confirmations or questions in doc comments
4. Start executing tasks handed off from Beijing

Reverse direction same process. US does same handoff before leaving.

```
Feishu Bot: "Today's Handoff" Auto Template

━━━━━━━━━━━━━━━━━━━━━━━━━━━━
□ Beijing → US Handoff | 2026-04-22

□ Completed Today
• [PR #234] Payment module refactor merged
• [Doc] New API docs updated
• Customer A's bug fixed, pending verification

□ Need US To Do
• @Tom Please review PR #235 (DB migration)
• @Sarah Customer B has demo tomorrow, confirm env
• In Base "Backlog", P0-03 needs your effort estimate

⚡ Decision Log
• Decided on Option B for cache layer (rationale in decision log [link])
• If you disagree, comment in doc

□ Blockers
• AWS permissions not ready yet, need Tom to help apply

□ Context
• Option B takes 2 days more than A, but 60% lower long-term maintenance
━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

> ***Key Practice**: More detailed handoff = higher teammate efficiency Good handoff vs bad handoff. Good: teammate reads for 5 minutes and starts working. Bad: "Did some stuff today, take a look." Look at what? Where? What needs to be done? 12 hours wasted.*

04.4 Cross-Timezone Bot Automation

These flows run in n8n, executed via Feishu Bot:

AUTOMATION	TRIGGER TIME	ACTION	PUSH TO
Beijing Morning	08:00 UTC+8	Aggregate US overnight updates	#daily-brief
US Morning	09:00 UTC-7	Aggregate Beijing daytime updates	#daily-brief
Beijing Daily	17:00 UTC+8	Beijing summary + handoff	#daily-brief
US Daily	17:00 UTC-7	US summary + handoff	#daily-brief
PR Reminder	On PR creation	Notify reviewer (timezone-aware)	#engineering
Cross-timezone Todo	When @ other timezone	Add to their morning brief	Auto
Timeout Alert	Task no response 24h	Escalation reminder	Task Owner

> ***Timezone-aware urgency*** *Common pitfall: Send urgent message to US colleague at 1 AM Beijing (10 AM US). Lark urgent notification keeps ringing on their phone. Correct approach: Non-urgent messages use regular channels, Bot batches for their work hours. Only "system down" level issues use urgent.*

04.5 Cross-Timezone Conflict Resolution

Most common conflict: Beijing made Decision A, US wakes up thinking B should be done.

Resolution flow:

```
1. US writes objection and rationale in decision log doc comments
2. 24-hour async discussion window (both sides comment in doc)
3. If consensus → Update decision log
4. If no consensus → 15-min Lark Meeting in overlap window
5. Final decision authority? Define per decision type in SOP
```

DECISION TYPE	DECISION OWNER	OTHER TIMEZONE ROLE
Product direction	CEO/Product Lead	Async input, Owner decides
Technical architecture	CTO/Tech Lead	Async discussion, Owner decides
Client relationships	Timezone closest to client	Other timezone provides support
Emergency incident	On-duty timezone	Fix first, retro later

> ***Core Practice**: Cross-timezone's worst enemy: "Waiting for confirmation before starting" If every item waits for confirmation, each back-and-forth takes 12 hours, only 3-4 iterations per week. Make bold decisions, document rationale thoroughly. Adjust if they object. 5x faster than "waiting."*

Part 2: Daily Operations SOP

How the team runs daily, weekly, monthly.

---

§ 05 Daily Communication: Bot + Async Messages Replace 80% of Meetings

05.1 Daily Standup is Dead

Daily standup is impossible in cross-timezone teams — can you find a time all 5 people are available? You can't.

I did the math: Even without timezone issues, 15 min × 5 people × 250 days = 312.5 person-hours/year. Feishu Bot does the same in 0 minutes.

05.2 Bot Daily Brief Replaces Standup

n8n + Feishu Bot implementation:

```
Daily 08:00 UTC+8 auto-triggers "Beijing Morning Brief":
1. Pull yesterday's completed tasks from Lark Projects
2. Pull yesterday's PRs and merges from GitHub
3. Pull new tickets from Lark Base "Customer CRM"
4. Pull anomalies from monitoring system
5. Extract todos from US overnight handoff doc
6. AI (Claude API) generates structured summary
7. Feishu Bot pushes to #daily-brief group
```

Bot message format:

```
 Beijing Morning Brief | 2026-04-22 Wed

── Yesterday's Wins ──
• [Product] Registration flow optimization live (@Jialu)
• [Eng] API rate limiting bug fix merged (@Alice)
• [Growth] Newsletter sent, 32% open rate (@Bob)

── Handoff from US ──
• @Tom Reviewed PR #235, 2 suggestions need changes (see PR comments)
• @Sarah Customer B demo environment confirmed OK
• AWS permissions applied, expected today

── Today's Priorities ──
1.  Payment failure rate up 2% yesterday, needs investigation
2.  Feature Y prototype review (doc ready, async comments please)
3.  Content: Prep next week's SEO article batch

── ⚠ Watch List ──
• Customer A contract expires Friday, follow up on renewal
• AWS costs projected 15% over budget this month

── Data Snapshot ──
DAU: 2,340 (↑3.2%) | New signups: 89 (↓5%)
Customer tickets: 12 (Bot handled 10, 2 need human)
```

> ***Key point**: This Daily Brief generates with 0 human effort. Beijing team spends 3 minutes reading. Respond in thread if needed. 10x more information than 15-min standup, 90% less time cost. Cross-timezone teammates offline? No problem.*

05.3 Structured Message Format

When posting work messages in Feishu groups, use structured format:

```
 [Request/Update/FYI/Decision]

Title: XXXXX
Details: Detailed description
Who needs to do what: @Jialu Review PRD | Due: Today 18:00 UTC+8
Related doc: [Lark Doc link]
Cross-timezone note: If US has input, leave in doc comments
```

Bot can auto-parse this format, extract tasks to Base, track deadlines.

05.4 Three Async Communication Methods

Method 1: Doc Comments Replace Discussion Meetings

Need input on a proposal? Write a Lark Doc, @ stakeholders, set 24-hour deadline. Everyone comments in doc. After 24h, Bot summarizes all comments, extracts consensus and disagreements.

Why 24 hours? This covers one full workday across both China and US timezones.

Method 2: 5-Min Video Replaces 30-Min Meeting

Need to demo a feature or explain a proposal? Record 5-min Lark video. Teammates watch anytime, AI auto-transcribes. Cross-timezone teammates don't wait for you to be online.

Method 3: Lark Base Replaces Status Reports

Don't ask "where is XX at?" in chat. Check Base. Status updates real-time. Bot auto-pushes status changes.

> ***Golden Rule of Cross-Timezone Communication*** *Before sending any message, ask: If the recipient sees this 12 hours later, can they act without asking me? If not, add more context.*

§ 06 Weekly Meetings & Decisions: How to Run the One Meeting Worth Having

06.1 One Meeting Per Week

AI-native startups run **one formal meeting per week: Monday Weekly**.

Cross-timezone scheduling: Use overlap window.

BEIJING TIME	US WEST TIME	DURATION
Monday 08:30-10:00	Sunday 17:30-19:00	90 min

> ***Note**: US is Sunday evening Not ideal. Alternative: Beijing Monday 21:00-22:30 = US Monday 06:00-07:30. Pick what both sides can accept. Key: Only once per week, endure it.*

If no good time works, there’s another option: **Fully Async Weekly**.

06.2 Sync Weekly Template

```
# Weekly | 2026-04-21 Mon 08:30-10:00 Beijing

## Pre-read (sent 24h before)
• Last week's decision log: [Link]
• This week's metrics dashboard: [Link]
• Each Pod's written update (max 5 lines per Pod)

## Agenda (strict 90 min)

### 1. Metrics Review (10 min)
- Bot presents: DAU, retention, revenue, key funnel metrics
- Only discuss anomalies

### 2. Decision Log Review (15 min)
- What decisions were made last week?
- Any objections raised async?
- Confirm or revisit

### 3. Pod Updates (30 min, 10 min each)
Each Pod lead:
• 3 accomplishments last week
• 2 priorities this week
• 1 blocker needing help

### 4. Cross-Timezone Handoff Review (10 min)
• Did handoffs work smoothly?
• Any information gaps?
• Adjust SOP if needed

### 5. This Week's Decisions (20 min)
• What needs deciding this week?
• Who owns each decision?
• Async or sync decision process?

### 6. Action Items (5 min)
• Bot captures todos in real-time
• Posted to #daily-brief within 10 min after meeting
```

06.3 Async Weekly Alternative

If sync weekly is impossible, run it fully async:

```
Monday 08:00 Beijing: Bot posts weekly template to #weekly
Each Pod lead fills their section by Tuesday 18:00 Beijing
Wednesday: Async discussion in doc comments
Thursday: Bot summarizes, flags decisions needing sync discussion
Friday: If needed, 30-min sync meeting for critical decisions only
```

> ***90% of weeklies can be async*** *Only discuss what can't be resolved in writing. Most metrics review, updates, and routine decisions work perfectly async.*

06.4 Decision Framework

Every decision needs:

```
**Decision**: What was decided
**Rationale**: Why this option
**Data**: Supporting evidence
**Decision-maker**: Who made the call
**Stakeholders**: Who was consulted
**Implementation**: Next steps
**Review date**: When to revisit
```

Document in Decision Log immediately. Bot notifies stakeholders.

Part 3: Pod Operations

How small autonomous teams run.

---

§ 07 Pod Structure: Small Teams + AI Agents

07.1 What is a Pod

First heard "Pod" in late 2024. Thought it was another Silicon Valley buzzword. Then tried it. Completely changed my mind.

A Pod is a small cross-functional strike team. 3-5 people + AI Agent. Organized around a clear mission.

Key design principle for cross-timezone Pods: **Keep Pod members in the same timezone if possible.** If must be cross-timezone, use relay baton protocol.

07.2 Pod Composition (10-Person China-US Team Example)

POD	BEIJING MEMBERS	US MEMBERS	AI AGENT	COLLABORATION MODEL
Product Pod	Full-stack×1 + PM×1	Full-stack×1	Code/Test/Doc Agent	Relay: Beijing does frontend, US does backend
Growth Pod	Growth Lead×1	Content×1	Content/SEO/Data Agent	Beijing sets strategy, US handles English markets
Infra Pod	DevOps×1	Full-stack×1	Monitor/Deploy Agent	24/7 coverage: Rotating on-call
Founder Layer	CEO×1	CTO×1	Strategy/Finance Agent	Overlap sync + rest async

> ***Key insight**: Infra Pod design Beijing + US one person each, naturally creates 24/7 coverage. Beijing daytime on-call, US daytime on-call. System always watched. Cross-timezone becomes advantage.*

07.3 Lark Projects + Base Task Management

Lark Projects for Sprint management, Base for data tracking.

1-week Sprint. Monday sets priorities, Friday reviews results.

Sprint flow (cross-timezone):

```
Monday Weekly: Align this week's priorities (sync or async)
Monday-Friday: Execute (async collaboration + Bot)

Daily 08:00 UTC+8: Bot pushes Beijing Morning Brief
Daily 09:00 UTC-7: Bot pushes US Morning Brief
Daily 17:00 each timezone: Bot pushes Daily Report + Handoff

Friday 17:00 UTC+8: Bot auto-generates Sprint Review
├── Completion rate
├── Incomplete items and reasons
├── Cross-timezone collaboration quality score
├── Next week's suggested priorities
└── Push to Lark Doc
```

Auto status update (n8n + Lark API + GitHub Webhook):

```
Code push → Lark Project task auto-becomes "In Development"
PR created → Auto-becomes "Code Review" + Bot notifies reviewer
PR merged → Auto-becomes "Pending Deploy"
Deploy success → Auto-becomes "Completed" + Base record updates
```

> ***Core Practice**: Slice Pods by feature modules Don't make the same feature need Beijing and US developers alternating. Code handoff cost is too high. Let Beijing Pod own Feature A entirely, US Pod owns Feature B entirely. Interface only at API layer.*

Part 4: Business Workflows

Product, Engineering, Growth — how each runs.

---

§ 08 Product Workflow: 4-Day Cycle from Idea to Live

08.1 Traditional Product Flow is Too Slow

Traditional flow: Write PRD (3 days) → Review (1 day) → Design (3 days) → Dev (5-10 days) → Test (3 days) → Release (1 day). Total: 16-21 days.

AI-native teams can do 4 days. Cross-timezone adds advantage — 24/7 continuous operation. Beijing works half, hands to US to continue.

08.2 4-Day Delivery Cycle (Cross-Timezone)

```
Day 1: Requirements + Design
├── Beijing morning: AI analyzes user feedback (Lark Base auto-aggregate)
├── Beijing AM: PM writes half-page requirement summary (Lark Doc)
├── Beijing noon: AI generates full PRD draft + prototype
├── Beijing PM: Team async comments in doc (24-hour window)
├── Beijing evening: PM finalizes, writes handoff to US
└── US morning: US engineer reviews PRD, comments in doc

Day 2: Development
├── Beijing morning: Review US comments, resolve PRD issues
├── Beijing all day: AI Pair Programming (core modules)
├── Beijing evening: Handoff to US (PR + context)
└── US all day: Continue dev (auxiliary modules + integration)
    → Leverage timezone gap for near 24-hour continuous dev

Day 3: Test + Optimize
├── Beijing morning: Review US commits, AI runs full test suite
├── Beijing all day: Fix bugs + manual edge case testing
├── Beijing evening: Code Review + handoff
└── US morning: Final review + merge

Day 4: Release + Monitor
├── Beijing morning: Auto-deploy to canary
├── Beijing all day: AI monitors canary metrics
├── Beijing evening: Metrics normal → full release
└── US: AI Agent continuous overnight monitoring
    → 24/7 monitoring coverage
```

> ***Hidden advantage of cross-timezone*** *Many see timezone as disadvantage. In AI-native product dev, it's advantage. Beijing works, hands to US to continue, team effectively runs 24 hours. 4-day cycle has 50% more effective dev time than same-timezone teams.*

08.3 Requirement Analysis: AI + Feishu Workflow

Lark Base "Backlog" auto-aggregate:

```
├── Customer support group → Bot extracts feedback → Write to Base
├── User behavior data → AI analyzes → Write to Base
├── Competitor monitoring → AI summarizes → Write to Base
└── Internal team → Add to Base anytime

      ↓ AI auto-prioritizes

  Backlog Base Views
  ├── P0 View: Must do this week
  ├── P1 View: Do next week
  ├── P2 View: Queue
  └── Dashboard: Source distribution, priority distribution
```

> ***Lark Base is your requirement hub*** *Don't say in chat "I think we should build XX feature". Add a record to Base. Attach data if you have it. PM picks this week's work from Base. AI helps prioritize, PM makes final call.*

---

§ 09 Engineering Workflow: AI Pair Programming Daily

09.1 AI Programming in 2026 is Not "Assistance"

I've used AI for coding for 2 years. From "complete a line" to "I describe requirements, it handles implementation." Change happened faster than expected.

Cursor: 20 engineers → $1B ARR. One of the highest revenue-per-employee software companies ever.

09.2 Engineer’s Daily Workflow

```
08:30  Read Bot morning brief + US handoff doc
09:00  Open Claude Code / Cursor
09:15  Start first task
       ├── Describe requirements in natural language
       ├── AI generates code framework
       ├── I make architectural decisions + review
       ├── AI implements details
       └── AI auto-writes tests

11:00  Submit PR → Bot auto-notifies reviewer
11:15  Start second task

13:30  Review others' PRs
       ├── AI does Code Review first, flags issues
       ├── I check AI flags + overall reasonableness
       └── Write review comments in doc or GitHub

16:00  AI runs full test suite → Fix issues
17:00  Write handoff doc → Bot pushes daily report
```

09.3 Cross-Timezone Code Review Practice

Cross-timezone Code Review key: **AI reviews first, humans second, don’t wait.**

```
Step 1: PR created → AI auto-review (2 min)
├── Code style, bugs, security, performance, test coverage
└── Report pushed to PR comments + Feishu #engineering

Step 2: Same-timezone reviewer (if any) → Review same day
        Cross-timezone reviewer → Review when they start work

Step 3: Don't Block: If AI Review passes and no critical issues
         → Can merge to staging first
         → Human review before merge to main
```

> ***Don’t let PRs wait cross-timezone for >24 hours*** *If a PR needs cross-timezone reviewer, max one business day. PRs waiting >24 hours auto-escalate via Bot.*

09.4 Model Selection

SCENARIO	RECOMMENDATION	WHY
Daily coding	Claude Code (Sonnet)	Highest code quality
Rapid prototyping	Cursor + GPT-4	Speed
Code Review	Claude Sonnet	High accuracy
Test generation	Cheaper models for cost-sensitive	Save money
Documentation	Claude Haiku	Good enough, cheap

§ 10 Growth Workflow: 2 People Doing 20 People’s Work

10.1 AI-Native Growth Core Logic

I’ve done growth. I know how resource-intensive traditional growth is. A 5-person marketing team, writing content, sending emails, chasing data. This approach is ending.

Cross-timezone growth has natural advantage: **Chinese and English markets can run simultaneously.**

10.2 Growth Workflow (China-US Dual Market)

ROLE	LOCATION	RESPONSIBLE FOR	AI AGENT SUPPORT
Growth Lead	Beijing	Strategy + Chinese market + Data	Data analysis Agent
Content	US	English market + Social + SEO	Content gen Agent + SEO Agent

```
Weekly Growth Cycle:

Monday: Bot pushes Growth Weekly Report (auto-generated)
├── Chinese market data + US market data
├── Channel performance ranking
├── Competitor moves
└── AI suggestions for this week's focus

Monday-Tue: Content Production
├── Beijing: AI generates Chinese content → Growth Lead reviews
├── US: AI generates English content → Content reviews
├── Lark Base "Content Calendar" unified management
└── Bot auto-schedules publishing

Wed-Thu: Distribution & Outreach
├── AI auto-sends personalized emails
├── AI auto-engages on social media
├── High-value client 1v1 (handled by person in that timezone)
└── Bot monitors metrics

Friday: Retro
├── Bot generates weekly growth report → Lark Doc
├── Growth Lead adjusts strategy
└── Update Lark Base "Experiment Log"
```

10.3 Content Production SOP

STEP	WHO	HOW	TIME
Topic selection	AI suggests + Human decides	AI analyzes search data, recommends to Base	Human review 30 min
Outline	AI generates + Human adjusts	AI generates → Human adjusts in Lark Doc	5 min per article
First draft	AI	Based on outline + brand style guide	Auto
Review	Human	Check facts, tune tone, add experience	15 min per article
Publish	AI	Auto-publish + distribute	Auto

> ***Create brand style guide docs in Lark Docs first*** *Style guide includes tone, sentence preferences, banned words, target reader persona. One for Chinese, one for English. With these docs, AI content quality improves dramatically.*

---

§ 11 Customer & Operations Workflow: AI Agents on the Front Line

11.1 Cross-Timezone Advantage for Customer Support

I’ve done customer support personally. Responding to 100 emails daily — don’t want to go back.

Cross-timezone teams have natural advantage: **24-hour response coverage.**

11.2 Customer Support Workflow

```
Customer submits issue
    |
    ▼
Feishu Bot + AI Agent First Layer
├── Auto-classify (technical/usage/complaint/suggestion)
├── Auto-match knowledge base
├── Generate reply draft
|
├── Solvable (80%) → Reply directly
|   └── Write to Lark Base "Customer CRM"
|
└── Unsolvable (20%) → Escalate
    ├── Push to Feishu #customer group
    ├── Include: Customer profile + history + AI analysis + suggested reply
    ├── On-duty timezone person handles
    └── Solution syncs back to knowledge base (AI handles next time)
```

11.3 Operations Automation

OPERATION TASK	AI DOES	FEISHU TOOL	HUMAN DOES
Bookkeeping	Auto-classify transactions	Lark Base	Monthly review
Data reports	Auto-generate daily/weekly/monthly	Bot + Doc	Read conclusions, make decisions
Calendar mgmt	Auto-schedule with timezone consideration	Lark Calendar	Confirm important meetings
Approvals	Auto-route	Lark Approvals	Make approval decisions
Email	AI classify + auto-reply	Lark Mail	Handle important emails

> ***Operations is easiest to AI-automate — start here*** *Operations is standardized, fault-tolerant. Master operations first, accumulate AI workflow experience, then transform product and engineering.*

Part 4: Advanced Operations

Optimizing after you survive.

---

§ 12 Onboarding: AI-Native 1-Week Ramp-Up

12.1 AI-Native Onboarding (Cross-Timezone)

```
Day 1 (AM): Environment Setup
├── Bot sends welcome message + permission checklist
├── Self-service: Feishu suite + GitHub + AI coding tools
├── Lark Projects auto-creates new hire task board
└── Read "Knowledge Base/Onboarding Guide" (30 min)

Day 1 (PM): 1v1 with Founder (Lark Meeting)
├── 30 min: Mission, stage, top 3 priorities
├── Cross-timezone team intro + collaboration norms
└── If cross-timezone: Assign buddy in other timezone

Day 2: AI-Guided Learning
├── Bot recommends doc reading order based on role
├── Comprehension quiz after each doc (Lark Base form)
├── New hire introduces in #random
└── Start observing group chats and doc discussions

Day 3: First Small Task
├── Assign low-difficulty real task
├── Complete with AI Pair Programming
├── Someone reviews output
└── Cross-timezone: Buddy leaves feedback in handoff doc

Day 4-5: Independent Work
├── Take real Sprint tasks
├── Join Pod
├── Ask AI (knowledge base) first, humans second
└── Attend first weekly meeting

End of Week 1:
├── Complete at least 1 full task
├── Bot generates "Week 1 Report" → Lark Doc
└── Founder 5-min async feedback (Lark video)
```

> ***Knowledge layer quality determines onboarding speed*** *Once hired an engineer who submitted first PR on Day 3. Not because he was exceptional. Because Feishu knowledge base was complete enough. Bot answered 80% of his questions. Cross-timezone buddy left detailed context in handoff doc. He barely waited for anyone.*

12.2 Interview Screening

DIMENSION	TEST METHOD	RED FLAG
AI collaboration	Task + AI tools, 2-hour limit	Can’t do prompt iteration
Self-drive	Review project history	Never completed project independently
Async adaptability	Direct question	“I prefer face-to-face”
Cross-timezone exp	Check past experience	Never worked remote
Written communication	Review communication history	Can’t write clear context

> ***Written communication is hard requirement for cross-timezone teams*** *Someone who can’t write clear handoff docs is disaster in cross-timezone teams. Every handoff requires follow-up, every follow-up waits 12 hours. During interview, ask candidate to write a “project handoff note” — see if they can explain things clearly.*

---

§13 Security Guardrails & Agent Permissions: Draw Lines on Day 1

13.1 A Lesson

Almost made similar mistake myself.

End of 2025, a friend’s AI-native company: Customer support Bot wrote sensitive customer info into public Lark Doc. Because Bot wasn’t configured with data boundaries. Took 2 weeks to fix.

13.2 Feishu Bot Permission Matrix

BOT/AGENT	CAN READ	CAN WRITE	CANNOT ACCESS	HUMAN REVIEW NODE
Daily Brief Bot	Lark Projects, GitHub, monitoring	#daily-brief group	Customer private data	None (push only)
Customer Support Bot	Knowledge base, FAQ	Customer ticket replies	Customer payment info	20% escalate to human
Content Agent	Brand guide, SEO data	Content draft docs	Customer data	Human final review before publish
Code Agent	Codebase, tech docs	Code branches	Production env	PR needs human review
Data Agent	Business data Base	Report docs	Personal info fields	Human review when involves decisions

13.3 Four Red Lines

Red Line 1: Customer data doesn’ t cross Feishu permission boundaries. Bot can read anonymized data, cannot write raw data to public docs.

Red Line 2: Production deployment needs human button press. AI can deploy to staging, cannot auto-deploy to production.

Red Line 3: External content needs human review. Bot-generated customer emails, social content — human glance before sending.

Red Line 4: Spending needs human approval. AI suggests budget, cannot auto-spend. Use Lark approval flow.

> ***Spend 2 hours writing an AI usage policy, store in Lark Doc*** *1 page is enough. Write clearly what each Bot can/cannot do, who has permission to adjust, who to contact if issues. This doc can save you.*

---

§ 14 Scaling from 5 to 50: How Collaboration Evolves

14.1 The Scaling Trap

Observed too many AI-native startups crash during scaling.

Most common mistake: **Scaling with traditional hiring methods, losing async-first culture.**

Common pitfall: AI-native company scales from 10 to 40. New hires don't adapt to async ways, start pulling meetings, DM decisions. Old team rhythm disrupted. Productivity per person actually drops.

14.2 Scaling Checklist (China-US Cross-Timezone)

STAGE	HEADCOUNT	KEY ACTIONS
Seed	3-5	Build all §01-§11 SOPs + Feishu stack
Growth	5-15	Replicate Pods, complete Pods in Beijing and US
Expansion	15-30	Add AI ops role, standardize all Bots
Scale	30-50	Governance framework, security audit, add Europe timezone for triple shift

> ***Key insight at scale****: Add Europe timezone. With Beijing + US + Europe teams, achieve true 24/7 continuous operation. Each timezone 8 hours, relay baton never stops.*

14.3 Pre-Hiring Checklist

1. Can this be solved by adding a Feishu Bot/AI Agent?
2. If must hire, is Feishu knowledge base ready? Can new hire ramp in 1 week?
3. Can new hire pass AI collaboration test?
4. Can new hire adapt to async-first + cross-timezone work style?
5. Is new hire’s written communication sufficient?

All five pass, then hire.

14.4 Final Words

AI-native startups that thrive share one common trait:

They’re not “adding AI” to traditional orgs. They redesigned work from Day 1.

Async-first. Feishu Bot coordinated. Knowledge fully externalized. Small team, high leverage. Cross-timezone relay baton.

This playbook gives you complete operations manual from 0 to 1. Feishu stack setup, group design, Bot configuration, cross-timezone handoff, how to run meetings, task management, product development, coding, growth, customer service, onboarding, security, scaling — all here.

Now go build.

Appendices

A Feishu Tools Quick Reference

FEISHU TOOL	PURPOSE	AI INTEGRATION
Lark Messages	Daily communication	Bot API push/read
Lark Docs	Knowledge base/PRD/SOP	Bot API create/write
Lark Base	Backlog/Bug/CRM/Sprint	Bot API read/write records
Lark Projects	Task management	Bot API update status
Lark Calendar	Calendar management	Bot API create/remind
Lark Meetings	Weekly/emergency huddle	Minutes auto-transcribe
Lark Approvals	Budget/release approval	Bot triggers workflow

B Core SOP Template Checklist

SOP	SECTION	WHEN TO USE
Relay Baton Handoff	§ 04.3	Every day before leaving
Daily Brief	§ 05.2	Daily auto
Weekly Sync	§ 06.2	Every Monday
Weekly Async	§ 06.3	Weekly
4-Day Delivery Cycle	§ 08.2	Every Sprint
Content Production	§ 10.3	Weekly
Customer Support Escalation	§ 11.2	Always running
Onboarding	§ 12.1	Every hire
Emergency Response	§ 06.5	When incidents happen
Bot Permission Matrix	§ 13.2	Day 1 setup

C Cross-Timezone Quick Reference

SCENARIO	DO THIS	DON’T DO THIS
Need input	Doc comment + 24-hour window	Wait for them to come online for meeting
Made decision	Write decision log + handoff doc	Mention casually in chat
Urgent issue	Urgent message + on-duty timezone handles	Wait for other timezone
Code review	AI reviews first + cross-timezone within 24h	Let PR wait >24 hours
Feature dev	Slice by module, each timezone independent	Same module both timezones
Run experiments	Record in Base unified	Keep notes separately
Discuss proposal	Write clear doc + async comments	Say “let’s find time to chat”

D Common Mistakes & Solutions

MISTAKE	CONSEQUENCE	SOLUTION
Cross-timezone standups	Someone in midnight meeting	Bot Daily Brief replacement
Unclear handoff docs	12 hours wasted on clarification	Use handoff template + Bot check
DM decisions	Cross-timezone teammates unaware	Force public channels + decision log
Bot without permissions	Data leak	Draw permission matrix Day 1
Same module cross-timezone	Constant code conflicts	Slice Pods by module
Not using Lark Minutes	Other timezone misses meeting	All meetings enable Minutes
Wait for confirmation	Everything 12 hours slower	Bold decisions + thorough rationale
Too many tools	Fragmented information	Feishu ecosystem + 3 external tools

---

> ***Created by Jialu Cheng*** *| AI Native Coder · Indie Developer*
> *X: @Xialu8888*

AI-NATIVE STARTUP ORGANIZATION PLAYBOOK

Complete collaboration SOP for 5-person team with 16-hour timezone spread shipping product in 4 days. Async-first, Feishu Bot automation, and relay baton handoff protocol for China-US cross-timezone AI-native startups.